Zero-Cost Proxies Fail at the Top: Insights and a Remedy

Anonymous authors
Paper under double-blind review

Abstract

Zero-cost proxies (ZCPs) have enabled low-cost Neural Architecture Search (NAS) by removing the computational overhead from model training. While there is a strong correlation between ZCPs and model performance at the scale of entire search spaces, this does not necessarily translate to guiding the search to top performing architectures. In this paper, we conduct extensive benchmarking over state-of-the-art proxies in the TransNAS-Bench-101 setting and observe that the correlation decreases dramatically when reducing the space to the best architectures. Our findings suggest that metrics cannot reach high-end architectures on their own and require a secondary metric to constrain the space. We leverage the number of parameters as a discriminator metric in the genetic framework and introduce ParaDis, a low-cost NAS algorithm that remains orthogonal to ZCP design, with no prior over the search space. Experiments on multiple benchmarks confirm that ParaDis reaches competitive performance on a diverse array of tasks.

1 Introduction

Neural Architecture Search (NAS) has received increasing attention as it enables the design of neural network architectures without a human expert, but has long struggled to reduce its computational cost. Various methods introduced in the early days of NAS (Zoph & Le, 2017; Liu et al., 2018a) treat the performance of the candidate architectures as a label to train a predictor. These remain the gold standard for finding the best architectures (White et al., 2021; Schrodi et al., 2023), but are impractical in real-world applications due to the large computational overhead of training each candidate from scratch.

Several lines of work related to reducing the computational cost of NAS have emerged, such as one-shot NAS (Pham et al., 2018; Guo et al., 2020; Chu et al., 2021; Liu et al., 2018b) and zero-shot NAS (Mellor et al., 2021; Abdelfattah et al., 2021). In the latter, the training of candidate architectures is entirely foregone in favor of ad-hoc metrics - so-called zero-cost proxies (ZCPs) - that estimate the performance of the model in its freshly initialized state, using limited data over a few forward and backward passes. Methods that incorporate these proxies have achieved competitive results (Jiang et al., 2023; Peng et al., 2024) while reducing the computational load by several orders of magnitude.

However, zero-shot NAS has yet to offer the same performance as traditional NAS. Prior work (Kadlecová et al., 2024; Krishnakumar et al., 2022) has shown that proxies, while they can provide meaningful insights at similar model sizes, are ultimately biased towards high parameter count. An exhaustive search of the NAS-Bench-201 search space (Dong & Yang, 2020) on the CIFAR-10 task (Krizhevsky et al., 2009) using state-of-the-art proxy MeCo (Jiang et al., 2023) reveals that the top-ranked architecture consists solely of convolutions. This suggests that widespread parameter-free architectural choices, such as skip connections, are overlooked despite their proven benefits (He et al., 2015; Orhan & Pitkow, 2017).

In this work, we first investigate the behavior of ZCPs under diverse settings in the NAS-Bench201 (Dong & Yang, 2020), NAS-Bench-301 (Siems et al., 2020) and TransNAS-Bench-101 benchmark (Duan et al., 2021) and observe an over-reliance of ZCP evaluation on Spearman correlation. Indeed, while a high Spearman correlation indicates that the proxy can generally separate high-performance architectures from lesser ones, it fails to reliably distinguish between architectures of similar performance, such as in a subspace of good architectures. This suggests that on many tasks, ZCPs cannot reach top percentiles of the space unless the space is constrained, for instance, via pairing with another metric.

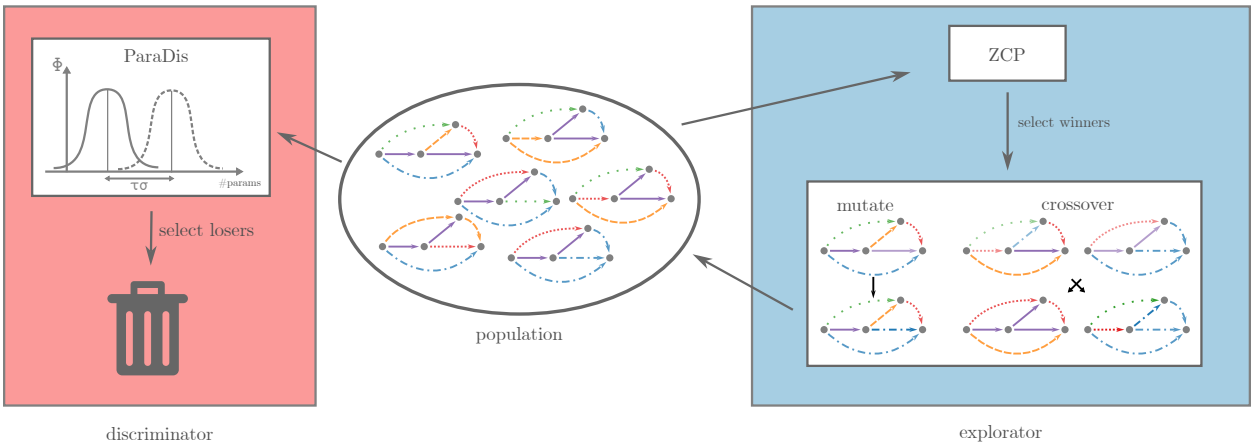


Figure 1: Our ParaDis method is based on the classic genetic zero-cost framework. Any ZCP can be used in the role of the explorer and select architectures that go on to reproduction. The ParaDis discriminator discards architectures based on parameter count distribution statistics over the population, with the size of architectures sampled at each generation controlled by τ.

We argue that parameter count is a simple yet universal metric to form such a pair. Other works (White et al., 2022; Javaheripi et al., 2022) have rightfully identified the parameter count of the model to be as strong as more complex ZCPs. However, it lacks the nuance to differentiate architectural choices at similar model sizes, which makes it a lackluster metric from the NAS point of view. Instead, we propose to view parameter count as a powerful constraint over the space to isolate more suitable regions for search via an associated ZCP. Moreover, parameter count is customizable as statistics over its distribution in the space can be directly used to target specific model sizes. We leverage this observation in **Para**meter **Dis**tribution Shift NAS (**ParaDis**), a novel NAS method based on a genetic algorithm framework, which assigns an arbitrary ZCP to the task of exploring the space, while simultaneously constraining based on parameter count (Fig. 1).

Our contributions are as follows:

1. We benchmark existing zero-cost proxies using the Trans-NAS-Bench-101 (Duan et al., 2021) benchmark and shed light on the shortcomings of Spearman-based ZCP evaluation. Specifically, we show that state-of-the-art proxies suffer from a drop in performance when confronted with a space of good architectures.
2. We introduce ParaDis, a genetic-based neural architecture search method that pairs ZCPs with parameter count to target good regions of the space. ParaDis is orthogonal to ZCP design, and makes no assumption about the search space.
3. We conduct extensive experiments on the NAS-Bench-201 and TransNAS-Bench-101 benchmarks to demonstrate the effectiveness of our method.

2 Related works

Zero-cost proxies Zero-cost proxies (ZCPs) (Mellor et al., 2021; Abdelfattah et al., 2021) hold the potential to conduct neural architecture search without the need to train architectures. A ZCP is generally derived from mathematical properties of neural networks computed from their parameters or gradients, which can be obtained during a single forward or backward pass on a batch of data. This enables the evaluation of a candidate architecture in seconds, as opposed to full retraining of the model.

Various ZCPs have been introduced for the purpose of architecture search. Initially, Mellor et al. (2021) proposed to use the Hamming distance of the binary codes of the activations, a proxy which was later renamed `jacov`. Abdelfattah et al. (2021) brought metrics from other fields into the NAS space. Interestingly, ZCPs

tend to fall into two categories based on whether they aim to quantify the trainability of the candidate - that is, how easily it can converge to its optimal weights - or its generalization capabilities. Thus, the metrics `gradnorm`, `snip`, `graph` and `synflow` are of the trainability type, while `fisher` is of the generalization type. This observation was further explored in Chen et al. (2021), who paired the trainability measure `ntk`, based on the NTK condition number (Xiao et al., 2019), and the expressivity measure `#lr`, a count of the number of linear regions of the input space partitioned by the network (Raghu et al., 2017).

Recently, more advanced ZCPs have emerged and demonstrated competitive results on vision tasks. `zen` (Lin et al., 2021) leverages the statistics of the batch norm layers. `zico` (Li et al., 2023) exploits the link between the statistics of the gradient norms and network generalization ability. `meco` (Jiang et al., 2023) estimate convergence rate via the minimum eigenvalue of the correlation matrix. `swap` (Peng et al., 2024) reframes the question of linear regions to create a more insightful metric. While these metrics show clear theoretical merit for search space exploration, we seek to enhance their performance through the search procedure.

Zero-shot NAS As ZCPs are only performance measures based on network properties, an adequate NAS algorithm is required to exploit them. Several possibilities have been explored in previous work. Mellor et al. (2021) proposed the naive random search solution, and warm-started other traditional NAS methods with the ZCP. Similarly, Abdelfattah et al. (2021) envisioned ZCPs as a warmup for other algorithms. Chen et al. (2021) and Jiang et al. (2023) implemented a pruning-like strategy where a directed acyclic graph containing all possible architectural choices is gradually pruned down to only the most desirable operation on each one of its edges.

However, the most popular search strategy in conjunction with ZCPs remains by far the genetic approach (Lin et al., 2021; Li et al., 2023; Peng et al., 2024). It is also a very common strategy for NAS in general (Real et al., 2018; Ericsson et al., 2024). Related zero-shot methods implement some form of evolution with elitist selection, meaning that ZCPs dictate which architectures are discarded from the population and which go on to mutation. In this paper, we also adopt the evolutionary framework. While other NAS methods keep the algorithmic design to a bare minimum, choosing to instead focus on the proxies' capabilities, we directly improve on the classic genetic algorithm to cover the limits of ZCPs. Importantly, this allows us to remain orthogonal to ZCP design.

ZCP ensembling Intuitively, using more than one proxy should be beneficial, since metrics that evaluate different aspects of a network might cover each other's blind spots. Chen et al. (2021) first introduced this idea by summing their `ntk` and `#lr` metrics. He et al. (2024) generalized the idea by ensembling 6 ZCPs through weighted linear combination and learning the weights through Bayesian optimization. However, direct summation of ZCPs is impractical as the statistics of individual proxies are task-dependent and largely unknown. Without robust normalization, such sums are highly sensitive to outliers. Kadlecová et al. (2024) instead of ensemble proxies by treating them as features of the candidate architectures and applying random forest regression. Differently, we assign roles to ZCPs within the genetic framework such that they guide exploration of the search space or discriminate architectures to constrain the space.

Parameter count Simply counting the parameters of a candidate architecture is a naive, yet effective proxy for accuracy. Indeed, network complexity and generalization abilities grow with network size. More often than not, the surprising effectiveness of the parameter count proxy `#params` often discredits ZCPs (White et al., 2022; Javaheripi et al., 2022), which are shown to be inferior to merely counting the parameters on various tasks. This metric lacks the nuance to discern common architectural choices such as the presence of skip connections, a staple of architecture design shown to improve trainability (He et al., 2015; Orhan & Pitkow, 2017). Complex ZCPs might be able to identify these intricacies while failing to describe the bigger picture - which `#params` does.

We propose that the parameter count may serve as a prerequisite metric and adopt it as a regularizer of the search. Peng et al. (2024) also used parameter count to regularize the score of their metric `swap` and focus the search around a desired model size. However, this requires prior knowledge of the search space's statistics which is impractical for unseen spaces where the location of good architectures in the distribution is

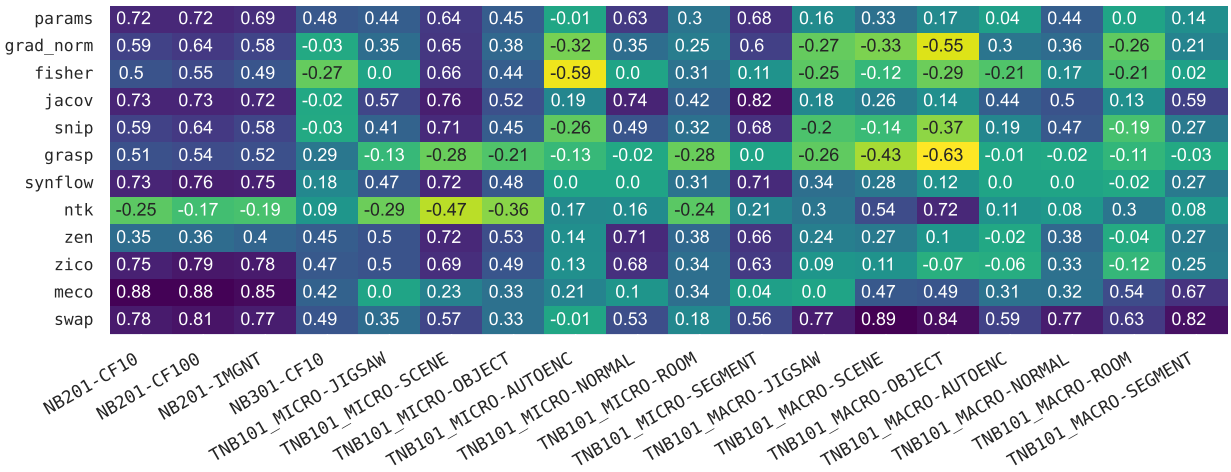


Figure 2: Spearman rank correlations of various ZCPs on the NAS-Bench-201, NAS-Bench-301 and TransNAS-Bench-101 benchmarks. Spearman was computed over 3 seeds against the entire search space.

unknown. We instead estimate the space's statistics directly from available sampled architectures, removing the need for priors over the space.

3 Investigating the limitations of ZCPs

3.1 Context and benchmark description

Intuitively, zero-cost proxies perform better the more correlated to actual network performance they are. In the situation of a head-to-head between several candidate architectures - the most commonplace situation for both the genetic and pruning approach in zero-shot NAS - a well-correlated metric is able to correctly choose the winner, which progresses the search in the direction of the more favorable architectures. Therefore it is without surprise that the literature has turned to benchmarking ZCPs based on their correlation to the most common search objective, validation accuracy.

A number of classic ZCPs have been benchmarked by Krishnakumar et al. (2022) on a collection of 5 search spaces (Ying et al., 2019; Dong & Yang, 2020; Siems et al., 2020; Duan et al., 2021). In this work, we consider only 4 of these search spaces:

- NAS-Bench-201 (Dong & Yang, 2020) (NB201), a cell-based space of 15625 architectures for image classification trained on CIFAR-10, CIFAR-100 and ImageNet16-120.
- NAS-Bench-301 (Siems et al., 2020) (NB301), a search space based on DARTS (Liu et al., 2018b) and evaluated on CIFAR-10 via the surrogate method.
- TransNAS-Bench-101 Micro (Duan et al., 2021) (TNB101-Micro), a cell-based search space of 4096 architectures tailored for diverse tasks such as object detection in the Taskonomy task bank (Zamir et al., 2018) or autoencoding tasks (Krishnakumar et al., 2022).
- TransNAS-Bench-101 Macro (Duan et al., 2021) (TNB101-Macro), a similar space to TNB101-Micro but focused on the macro structure with a stackable cell in 3256 different configurations.

NB201 and NB301 are designed for image classification and compatible with the common CIFAR-10, CIFAR-100 and ImageNet16-120 tasks (Krizhevsky et al., 2009; Deng et al., 2009; Chrabaszcz et al., 2017), the TNB101 benchmark focuses on other paradigms and evaluating the performance of NAS across domains and tasks and outside of the classical cell-based framework. The metric used for the benchmark is the Spearman

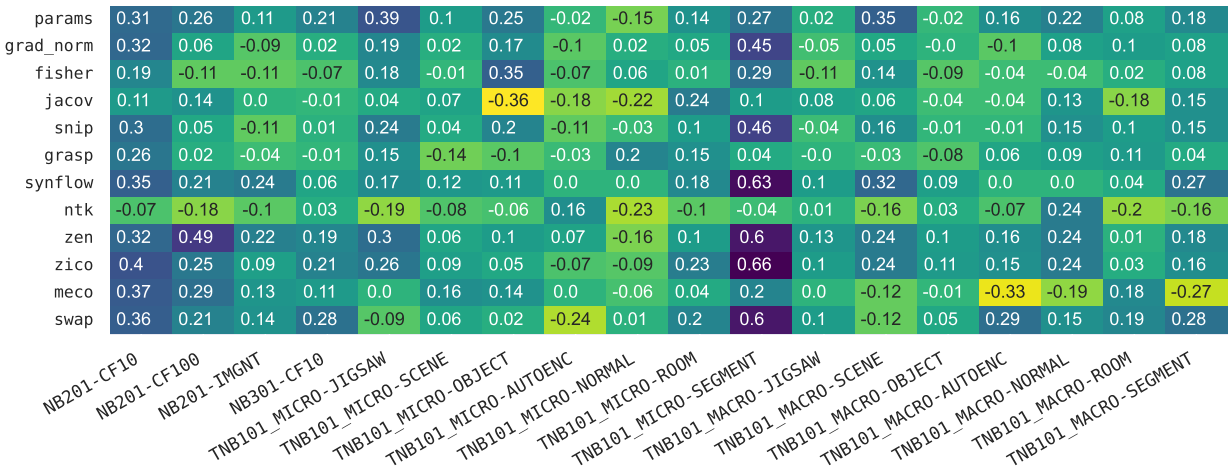


Figure 3: Spearman rank correlations of various ZCPs on the NB201, NB301 and TNB101 benchmarks. Spearman was computed on 3 seeds against the top 1% architectures in the space.

rank correlation which is the correlation of the ranks of the ZCP scores X and the ground truth scores Y:

$$r_s = \frac{cov(rank(X), rank(Y))}{\sigma_{rank(X)}\sigma_{rank(Y)}}$$

It indicates the average difference between the true rank of the candidate architecture and the rank given by the proxy. Proxies with high Spearman rank correlation tend to keep candidates ranked close to their true rank, such that a candidate is easily distinguished from others it outclasses by far.

3.2 Benchmarking: changing the perspective

We first reproduce the benchmark proposed by Krishnakumar et al. (2022) while including several state-of-the-art proxies that have been proposed since the initial benchmark. The results are shown in Fig. 8. The NAS-Bench-101 benchmark is omitted due to its large size while not providing significantly more insight than other image classification benchmarks. As we can see, newer proxies, namely `meco` and `swap`, reach high correlation on a variety of different tasks.

However, the represented search spaces are very wide and heterogeneous. We observe that on many tasks, the distributions of architecture performance in the TNB101 search spaces form distinct clusters. Therefore, high correlation over the entire search space only indicates that the proxy can eliminate all weak architectures and reach a high-performance cluster. Overall, the benchmark does not answer the central question of Neural Architecture Search: how easily can we reach the top architecture in the space?

In order to get the full picture, we adopt a novel perspective on benchmarking. For each search space, we run the benchmark restricted to only the top 1% of architectures in the space, in terms of validation accuracy. Assuming that proxies are good enough to guide the search to the top 1% cluster in the space, we want to assess whether they can also distinguish the performance variations in that group. The results of the benchmark are shown in Fig. 3. We notice a significant drop in correlation. On most tasks, none of the proxies have a correlation higher than 0.5, less than half of the correlation over the whole space. Some metrics, such as `meco` and `jacov`, show no correlation or negative correlation with the top architectures, despite having positive correlation with the entire space. Moreover, we observe that the ranking of metrics is shuffled around as metrics such as `zen` seem better suited to the top 1% benchmark than state-of-the-art proxies for the overall benchmark, such as `meco` and `swap`.

This analysis shows that while zero-cost proxies have demonstrated the ability to weed out poorly performing architectures during the search, they cannot properly rank architectures after having reached a sub-space

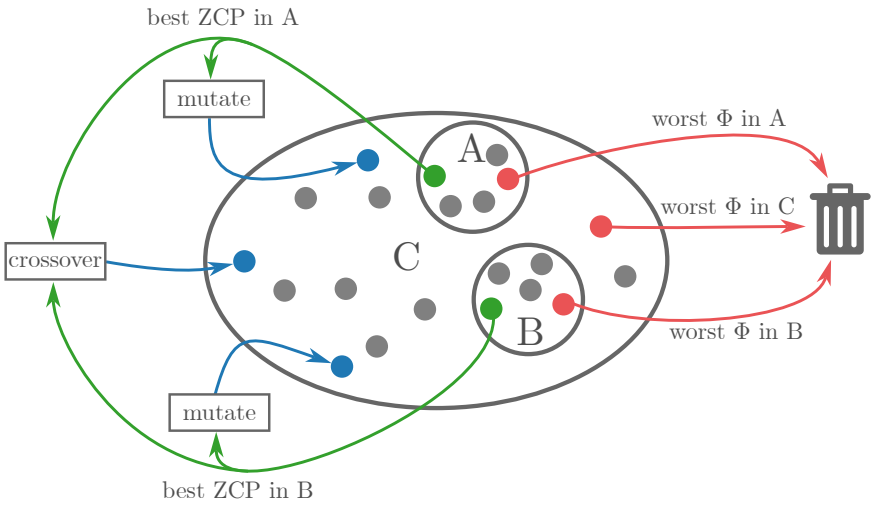


Figure 4: In the ParaDis genetic setup, architectures from the population are sampled into 3 groups. In each group, the explorer ZCP selects parents for reproduction, while the discriminator Φ removes ill-suited architectures. At each evolution step, 3 new architectures are created and 3 are discarded.

that contains only good candidates. Thus, considering the current state-of-the-art, there is a ceiling to the quality of architectures obtained through ZCP-driven search.

Interestingly, architectures within the same accuracy cluster, even among the top-performing ones, can differ significantly in structure. We hypothesize that using a second metric as a form of regularization of the search can constrain the search to a family of architectures that includes at least one top-performing model, and the proxy can find this architecture.

4 ParaDis: Parameter Distribution Shifted NAS

4.1 Genetic algorithm design

We introduce the ParaDis method, leveraging the genetic framework to conduct the search with a pair of metrics. For the basic genetic setup, we take inspiration from the elitist evolution algorithm proposed in Peng et al. (2024).

Candidate architectures are encoded into a genome in the form of a sequence of numbers, each corresponding to a possible architectural choice in the search space. While the genomes are processed and interpreted differently depending on the definition of the search space, they are processed identically within the genetic algorithm, thus decoupling search space design from the search algorithm. The various genome encodings for each search space are described in Appendix B.

At first, a number of random architectures from the search space are sampled to form the initial population. As the search progresses, architectures in the population are subject to mutation and crossover operations (Fig. 1). During mutation, a randomly selected operation choice is transformed into another non-zero operation. Crossover is also introduced to speed up the search by enabling larger leaps in the space. Two parent architectures evenly exchange genetic material to form a child with half of its architectural choices coming from each parent.

During each evolution step, the population is divided randomly into three groups: A, B, and C. A and B are of similar size, while C is made up of all remaining architectures. Competition takes place within groups A and B, where the least fit candidates in each group are removed from the population. The most fit candidates

are selected as winners: they undergo mutation and also serve as parents in the crossover operation. The resulting offspring candidate is added to the population, replacing the most unfit architecture in group C in order to maintain a constant population size. Therefore, at each evolution step, three new architectures are produced. This splitting setup introduces stochasticity to architecture selection, preventing the search from collapsing early into local optima due to the lack of genetic diversity.

We allow pairs of metrics to drive the search by assigning them two distinct roles:

- **Explorer** metric: directs exploration of the search by choosing competition winners, which go on to mutation and crossover. This should typically be a metric that can distinguish good architectures, even if there are false positives.
- **Discriminator** metric: constrains the space by choosing competition losers, which are removed from the population. This should be a very reliable metric that identifies architectures to weed out in order to restrict the space to a desirable region.

While we define these roles in relation with the genetic algorithm design, the main intuition - that some metrics are better suited to choose good architectures, and others to detect bad architectures - holds across other NAS paradigms.

Additionally, this framework is easily generalizable to the regular zero-cost setup by assuming the explorer and discriminator to be the same. However, most zero-cost proxies are designed for high-scoring architectures and are therefore more suited for the exploration role. For this reason, the explorer is flexible and can be any zero-cost proxy. This keeps the method fully orthogonal to proxy design. For the discriminator, we instead leverage the power of parameter count and introduce parameter distribution shift.

4.2 Parameter Distribution Shift

Several works have noted that the parameter count (`#params`) is a strong baseline for zero-shot NAS (White et al., 2022; Javaheripi et al., 2022). However, having an adequate number of parameters seems to be an obvious requirement for top architectures in the space, since those with low parameter counts are not complex enough to perform well on the tasks. On the other hand, previous works (Kadlecová et al., 2024; Krishnakumar et al., 2022) have shown that zero-cost proxies are themselves biased towards high parameter count, which provides an intuitive explanation for the results of our experiments in Section 3.2. While being biased towards higher parameter count guides the search well in general, the highest parameter count architectures are not the best, which prevents ZCPs that follow this behavior from reaching the top of the space.

Therefore, a clever use of parameter count as a discriminator is to define a score around distribution of parameter counts and target the right model size :

$$\Phi_\alpha = e^{-(\frac{p(\alpha)-\mu}{\sigma})^2}$$

where $p(\alpha)$ is the parameter count of the considered architecture, μ and σ are hyperparameters for calibration.

Interestingly, Peng et al. (2024) used this score to directly regularize their zero-cost proxy. However, due to the complete uncertainty over what kind of distribution parameters accurately represent the space, they chose μ and σ based on knowledge of the search space, which is unavailable in real-world contexts of NAS in unknown search spaces. Instead, we infer the parameters directly from the population's mean and standard deviation at the current generation, allowing the method to remain completely agnostic to the search space.

At the beginning of the search, encouraging the exploration of high-parameter-count architectures promotes population diversity. Toward the end, this bias is gradually reduced to favor architectures with lower parameter counts. To achieve this, the mean of the score distribution is adjusted according to a temperature-controlled schedule. The final score is computed as follows:

$$\Phi_{\alpha,\Pi}(\tau) = e^{-(\frac{p(\alpha)-(\hat{\mu}(\Pi)+\tau\hat{\sigma}(\Pi))}{\hat{\sigma}(\Pi)})^2}$$

where $\hat{\mu}(\Pi)$ and $\hat{\sigma}(\Pi)$ are the estimated mean and standard deviation over population Π, and τ varies from τ_{max} to τ_{min} during the search, both of which are search space-agnostic. The full genetic algorithm with parameter distribution shifted discriminator is detailed in algorithm Alg. 1.

4.3 Architecture selection

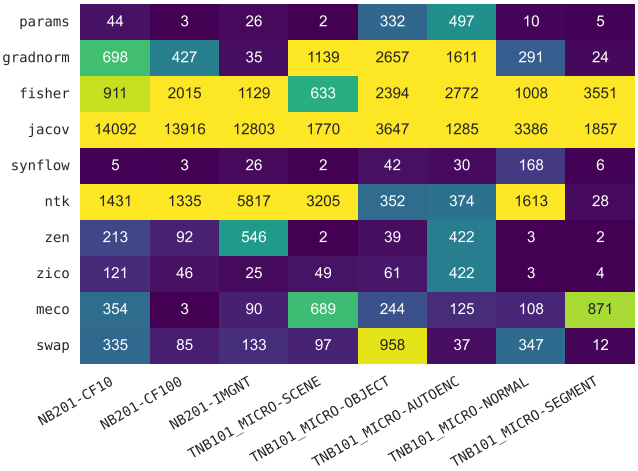
A key drawback of the genetic solution is its inability to produce a single architecture at the end of the search. In this section we propose a possible approach to tackle this issue. Under the assumption that following the search the space has been reduced to a very small number of candidate architectures, it becomes reasonable to train some of the candidates. Indeed, it is standard practice in NAS methods to retrain the obtained candidate architecture from scratch, therefore training counts slightly higher than 1 rather cost-efficient.

We propose to conduct the final selection of the candidates with successive halving (Hu et al., 2022; Li et al., 2018; Jamieson & Talwalkar, 2016). All candidates are trained jointly and at fixed milestones during the training, the bottom half of architectures is dropped until only one remains, which is trained to the max number of epochs to obtain the final, trained network. Considering a 600 epochs training, with a hypothetical milestone schedule in epochs such as $[10, 20, 50, 200]$ for a population of 25 architectures as in 5.1, the total cost of successive halving is 1400 epochs or 2.3 trainings.

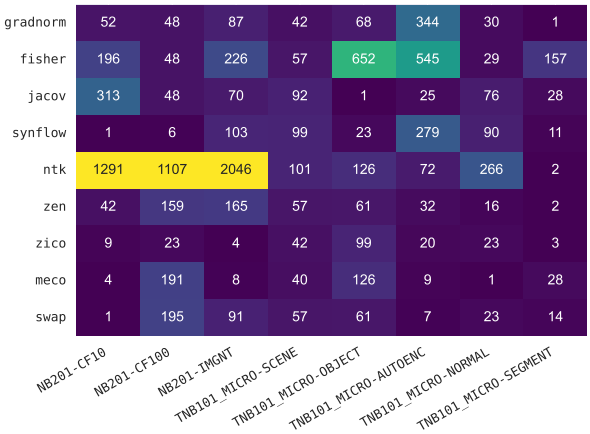
Previous work (Ly-Manson et al., 2024) has shown that validation accuracy is an unreliable metric for pruning networks in successive halving setups due to its inherent variability. Therefore, we propose to instead discard architectures at the given schedule based on the aggregated sum of all validation accuracies since training start, which is less volatile.

5 Experiments

Figure 5: Rank of best architecture in genetically evolved populations, on 8 different benchmark tasks and 10 ZCPs. We observe significant gains with a carefully designed discriminator.



(a) Genetic algorithm with a single ZCP



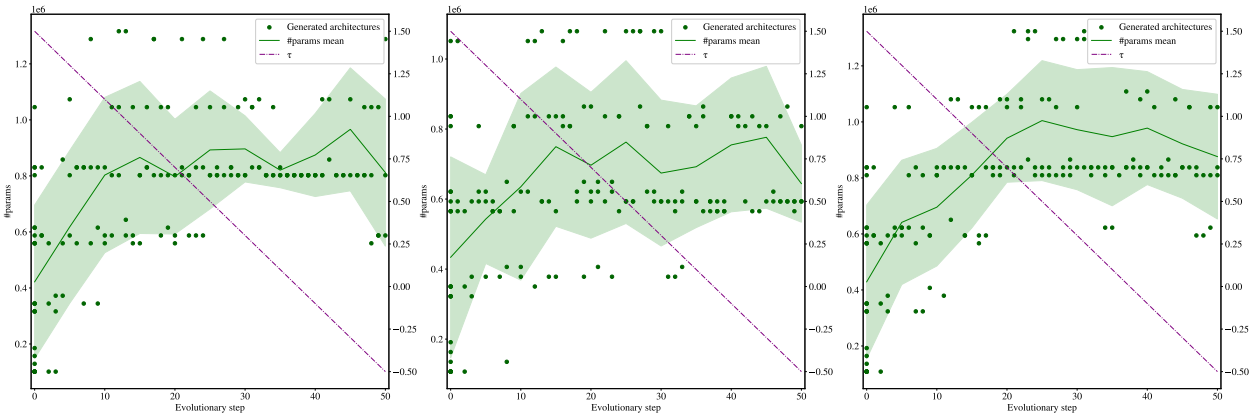
(b) Genetic algorithm with ZCP explorer and parameter-based discriminator

5.1 Benchmark

We apply our ParaDis method to the benchmarks described in 3.1, specifically 8 tasks from the NB201 and TNB101-Micro search spaces. We use the following hyperparameters for the genetic algorithm: population size 25, sample size of 8, 10 generations with 5 evolutionary steps per generation, totaling 50 evolution steps. For the ZCP evaluation, we set the batch size to 32 for the architectures.. On a single NVIDIA A100 GPU,

the running time of our method varies between 3 and 40 minutes depending on the task and the paired proxy. We report the ranks of the best architectures in the final populations for each search space and task in Fig. 5b. For the sake of comparison, we run our genetic algorithm as a regular zero-cost method (i.e, the discriminator metric is the same as the explorer's) using the same hyperparameters and report the results in Fig. 5a.

We observe a significant boost in the quality of architectures searched with the ParaDis discriminator compared to the single ZCPs. Specifically, the ranks of discovered architectures is lower on average for every metric, and the top-ranked architecture in the space is discovered for several metric/task pairs. Unsurprisingly, the biggest gains are obtained for ZCPs that struggle to reach a good region of the space on their own: on all tasks in the NB201 space, metrics such as `fisher` and `jacov` reach the top 2% of the search space. Overall, our benchmark experiments demonstrate that our parameter-based discriminator scheme restricts the search space to a sufficiently good region for all ZCPs to thrive.



(a) CIFAR10 (b) CIFAR100 (c) ImageNet16-120

Figure 6: Evolution of sampled architectures' parameter counts during genetic search with a parameter-based discriminator. The paired ZCP guiding the search is `meco`, which is usually biased towards overparameterized models. The average parameter count and its std error (in green) are constrained by the linear τ schedule during the search.

5.2 Model size control

To verify that the size of the models in the population is being meaningfully controlled by the ParaDis constraint, we plot the parameter counts of architectures sampled during the search. We present the results on three vision tasks in Fig. 6 with `meco`, a proxy heavily biased towards large model sizes, as the explorer metric.

We observe three easily identifiable phases of the search corresponding to the linear schedule of τ: at the start, the sizes of sampled architectures are pushed up by the explorer and the discriminator with positive τ. Towards the middle, the parameter counts stagnate or increase slowly as the population stabilizes and τ approaches 0. At the end of the search, with negative τ, the discriminator prevents parameter counts from increasing further, thus avoiding overparameterized regions. These results demonstrate that the discriminator properly controls the region of architecture sampling with respect to model size.

6 Limitations and perspectives

An important limitation of the family of zero-cost NAS methods is that, to the best of our knowledge, no universal solution exists. As observed in our experiments, no single ZCP is a clear cut above the others on every task, despite the similarities in domain and search space, although some metrics can be observed to perform better overall. Similarly, while the proposed ParaDis finds better architectures on average, it does

not perform better on every task. Therefore, a major drawback of the literature remains the overabundance of choice available between proxy design, search algorithm design and hyperparameters.

Additionally, while experimental results confirm the effectiveness of the genetic framework with the ParaDis discriminator, our proposed solution is merely a naive but effective approach. While the literature progresses rapidly on the topic of a proxy to guide the search towards better architectures, very little focus is given to proxies that select poor architectures to remove from the space. We hope that our insights will open this new question in the literature.

7 Conclusion

In this work, we identify a fundamental limitation of zero-cost proxies (ZCPs) in neural architecture search (NAS): their difficulty in distinguishing among top-performing architectures within large search spaces. While effective globally, ZCPs tend to favor architectures with high parameter counts, reducing their discriminative power at the high-performance end. To address this, we introduce ParaDis, a generic and search space-agnostic approach that controls the parameter count distribution to remove this bias. By constraining the search region, ParaDis enables ZCPs to better focus on promising candidates. Experiments on NAS-Bench-201 and TransNAS-Bench-101 demonstrate consistent improvements in discovered architectures across diverse tasks. Our method's robustness and versatility stem from its orthogonality to ZCP design and lack of assumptions on the search space. Overall, ParaDis advances zero-shot NAS by complementing ZCPs with an effective model size control mechanism.

References

Mohamed S Abdelfattah, Abhinav Mehrotra, Łukasz Dudziak, and Nicholas D Lane. Zero-cost proxies for lightweight nas. *arXiv preprint arXiv:2101.08134*, 2021.

Wuyang Chen, Xinyu Gong, and Zhangyang Wang. Neural architecture search on imagenet in four gpu hours: A theoretically inspired perspective. *arXiv preprint arXiv:2102.11535*, 2021.

Patryk Chrabaszcz, Ilya Loshchilov, and Frank Hutter. A downsampled variant of imagenet as an alternative to the cifar datasets. *arXiv preprint arXiv:1707.08819*, 2017.

Xiangxiang Chu, Bo Zhang, and Ruijun Xu. Fairnas: Rethinking evaluation fairness of weight sharing neural architecture search. In *Proceedings of the IEEE/CVF International Conference on computer vision*, pp. 12239–12248, 2021.

Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pp. 248–255. Ieee, 2009.

Xuanyi Dong and Yi Yang. Nas-bench-201: Extending the scope of reproducible neural architecture search. *arXiv preprint arXiv:2001.00326*, 2020.

Yawen Duan, Xin Chen, Hang Xu, Zewei Chen, Xiaodan Liang, Tong Zhang, and Zhenguo Li. Transnas-bench-101: Improving transferability and generalizability of cross-task neural architecture search. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 5251–5260, 2021.

Linus Ericsson, Miguel Espinosa Minano, Chenhongyi Yang, Antreas Antoniou, Amos J Storkey, Shay Cohen, Steven McDonagh, and Elliot J Crowley. einspace: Searching for neural architectures from fundamental operations. *Advances in Neural Information Processing Systems*, 37:1919–1953, 2024.

Zichao Guo, Xiangyu Zhang, Haoyuan Mu, Wen Heng, Zechun Liu, Yichen Wei, and Jian Sun. Single path one-shot neural architecture search with uniform sampling. In *Computer vision–ECCV 2020: 16th European conference, glasgow, UK, August 23–28, 2020, proceedings, part XVI 16*, pp. 544–560. Springer, 2020.

Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. corr abs/1512.03385 (2015), 2015.

Zhenfeng He, Yao Shu, Zhongxiang Dai, and Bryan Kian Hsiang Low. Robustifying and boosting training-free neural architecture search. *arXiv preprint arXiv:2403.07591*, 2024.

Shoukang Hu, Ruochen Wang, Lanqing Hong, Zhenguo Li, Cho-Jui Hsieh, and Jiashi Feng. Generalizing few-shot nas with gradient matching. *arXiv preprint arXiv:2203.15207*, 2022.

Arthur Jacot, Franck Gabriel, and Clément Hongler. Neural tangent kernel: Convergence and generalization in neural networks. *Advances in neural information processing systems*, 31, 2018.

Kevin Jamieson and Ameet Talwalkar. Non-stochastic best arm identification and hyperparameter optimization. In *Artificial intelligence and statistics*, pp. 240–248. PMLR, 2016.

Mojan Javaheripi, Gustavo de Rosa, Subhabrata Mukherjee, Shital Shah, Tomasz Religa, Caio Cesar Teodoro Mendes, Sebastien Bubeck, Farinaz Koushanfar, and Debadeepta Dey. Litetransformersearch: Training-free neural architecture search for efficient language models. *Advances in Neural Information Processing Systems*, 35:24254–24267, 2022.

Tangyu Jiang, Haodi Wang, and Rongfang Bie. Meco: zero-shot nas with one data and single forward pass via minimum eigenvalue of correlation. *Advances in Neural Information Processing Systems*, 36: 61020–61047, 2023.

Gabriela Kadlecová, Jovita Lukasik, Martin Pilát, Petra Vidnerová, Mahmoud Safari, Roman Neruda, and Frank Hutter. Surprisingly strong performance prediction with neural graph features. *arXiv preprint arXiv:2404.16551*, 2024.

Nitish Shirish Keskar, Dheevatsa Mudigere, Jorge Nocedal, Mikhail Smelyanskiy, and Ping Tak Peter Tang. On large-batch training for deep learning: Generalization gap and sharp minima. *arXiv preprint arXiv:1609.04836*, 2016.

Arjun Krishnakumar, Colin White, Arber Zela, Renbo Tu, Mahmoud Safari, and Frank Hutter. Nas-bench-suite-zero: Accelerating research on zero cost proxies. *Advances in Neural Information Processing Systems*, 35:28037–28051, 2022.

Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. 2009.

Jaehoon Lee, Lechao Xiao, Samuel Schoenholz, Yasaman Bahri, Roman Novak, Jascha Sohl-Dickstein, and Jeffrey Pennington. Wide neural networks of any depth evolve as linear models under gradient descent. *Advances in neural information processing systems*, 32, 2019.

Namhoon Lee, Thalaiyasingam Ajanthan, and Philip HS Torr. Snip: Single-shot network pruning based on connection sensitivity. *arXiv preprint arXiv:1810.02340*, 2018.

Guihong Li, Yuedong Yang, Kartikeya Bhardwaj, and Radu Marculescu. Zico: Zero-shot nas via inverse coefficient of variation on gradients. *arXiv preprint arXiv:2301.11300*, 2023.

Lisha Li, Kevin Jamieson, Giulia DeSalvo, Afshin Rostamizadeh, and Ameet Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18 (185):1–52, 2018.

Tengyuan Liang, Tomaso Poggio, Alexander Rakhlin, and James Stokes. Fisher-rao metric, geometry, and complexity of neural networks. corr. *arXiv preprint arXiv:1711.01530*, 2017.

Ming Lin, Pichao Wang, Zhenhong Sun, Hesen Chen, Xiuyu Sun, Qi Qian, Hao Li, and Rong Jin. Zen-nas: A zero-shot nas for high-performance image recognition. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 347–356, 2021.

Chenxi Liu, Barret Zoph, Maxim Neumann, Jonathon Shlens, Wei Hua, Li-Jia Li, Li Fei-Fei, Alan Yuille, Jonathan Huang, and Kevin Murphy. Progressive neural architecture search. In *Proceedings of the European conference on computer vision (ECCV)*, pp. 19–34, 2018a.

Hanxiao Liu, Karen Simonyan, and Yiming Yang. Darts: Differentiable architecture search. *arXiv preprint arXiv:1806.09055*, 2018b.

Timotée Ly-Manson, Mathieu Leonardon, Abdeldjalil Aissa El Bey, Ghouti Boukli Hacene, and Lukas Mauch. Analyzing few-shot neural architecture search in a metric-driven framework. In *International Conference on Automated Machine Learning AutoML24*, 2024.

Joe Mellor, Jack Turner, Amos Storkey, and Elliot J Crowley. Neural architecture search without training. In *International conference on machine learning*, pp. 7588–7598. PMLR, 2021.

A Emin Orhan and Xaq Pitkow. Skip connections eliminate singularities. *arXiv preprint arXiv:1701.09175*, 2017.

Barak A Pearlmutter. Fast exact multiplication by the hessian. *Neural computation*, 6(1):147–160, 1994.

Yameng Peng, Andy Song, Haytham M Fayek, Vic Ciesielski, and Xiaojun Chang. Swap-nas: Sample-wise activation patterns for ultra-fast nas. *arXiv preprint arXiv:2403.04161*, 2024.

Hieu Pham, Melody Guan, Barret Zoph, Quoc Le, and Jeff Dean. Efficient neural architecture search via parameters sharing. In *International conference on machine learning*, pp. 4095–4104. PMLR, 2018.

Maithra Raghu, Ben Poole, Jon Kleinberg, Surya Ganguli, and Jascha Sohl-Dickstein. On the expressive power of deep neural networks. In *international conference on machine learning*, pp. 2847–2854. PMLR, 2017.

Esteban Real, Alok Aggarwal, Yanping Huang, and Quoc V Le. Regularized evolution for image classifier architecture search.(2018). *arXiv preprint arXiv:1802.01548*, 2018.

Simon Schrodi, Danny Stoll, Binxin Ru, Rhea Sukthanker, Thomas Brox, and Frank Hutter. Construction of hierarchical neural architecture search spaces based on context-free grammars. *Advances in Neural Information Processing Systems*, 36:23172–23223, 2023.

Julien Siems, Lucas Zimmer, Arber Zela, Jovita Lukasik, Margret Keuper, and Frank Hutter. Nas-bench-301 and the case for surrogate benchmarks for neural architecture search. *arXiv preprint arXiv:2008.09777*, 4:14, 2020.

Hidenori Tanaka, Daniel Kunin, Daniel L Yamins, and Surya Ganguli. Pruning neural networks without any data by iteratively conserving synaptic flow. *Advances in neural information processing systems*, 33: 6377–6389, 2020.

Lucas Theis, Iryna Korshunova, Alykhan Tejani, and Ferenc Huszár. Faster gaze prediction with dense networks and fisher pruning. *arXiv preprint arXiv:1801.05787*, 2018.

Jack Turner, Elliot J Crowley, Michael O'Boyle, Amos Storkey, and Gavin Gray. Blockswap: Fisher-guided block substitution for network compression on a budget. *arXiv preprint arXiv:1906.04113*, 2019.

Chaoqi Wang, Guodong Zhang, and Roger Grosse. Picking winning tickets before training by preserving gradient flow. *arXiv preprint arXiv:2002.07376*, 2020.

Colin White, Willie Neiswanger, and Yash Savani. Bananas: Bayesian optimization with neural architectures for neural architecture search. In *Proceedings of the AAAI conference on artificial intelligence*, volume 35, pp. 10293–10301, 2021.

Colin White, Mikhail Khodak, Renbo Tu, Shital Shah, Sébastien Bubeck, and Debadeepta Dey. A deeper look at zero-cost proxies for lightweight nas. In *ICLR Blog Track*, 2022. URL `https://iclr-blog-track.github.io/2022/03/25/zero-cost-proxies/`. https://iclr-blog-track.github.io/2022/03/25/zero-cost-proxies/.

Lechao Xiao, Jeffrey Pennington, and Sam Schoenholz. Disentangling trainability and generalization in deep learning. 2019.

Huan Xiong, Lei Huang, Mengyang Yu, Li Liu, Fan Zhu, and Ling Shao. On the number of linear regions of convolutional neural networks. In *International Conference on Machine Learning*, pp. 10514–10523. PMLR, 2020.

Chris Ying, Aaron Klein, Eric Christiansen, Esteban Real, Kevin Murphy, and Frank Hutter. Nas-bench-101: Towards reproducible neural architecture search. In *International conference on machine learning*, pp. 7105–7114. PMLR, 2019.

Amir R Zamir, Alexander Sax, William Shen, Leonidas J Guibas, Jitendra Malik, and Silvio Savarese. Taskonomy: Disentangling task transfer learning. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 3712–3722, 2018.

Barret Zoph and Quoc Le. Neural architecture search with reinforcement learning. In *International Conference on Learning Representations*, 2017. URL `https://openreview.net/forum?id=r1Ue8Hcxg`.

A Search space encodings

In this section, we detail the encodings of the search spaces used for the experiments, largely based on the implementation from Krishnakumar et al. (2022).

A.1 NAS-Bench-201

The genome encoding of architectures from the NB201 space (Dong & Yang, 2020) is directly inferred from their graph-like cell structure. The cell is directly copied and stacked to form a complete network, therefore it is sufficient to encode a single cell. Each edge of the cell's graph is assigned to a genome position. The operation choices of the space (5 possible operations) are encoded into each position as a numerical value. With 6 edges in the graph, the resulting genome is a 6 values long string.

A.2 TransNAS-Bench-101 Micro

Due to being based directly on the same setting, the genome encoding of architectures for the TNB101-Micro space (Duan et al., 2021) is similar to the encoding for the NB201, with 6 edges and 4 operation choices.

A.3 TransNAS-Bench-101 Macro

The TNB101-Macro space (Duan et al., 2021) consists in a stack of 4 to 6 residual blocks where the locations of up to 4 downsamplings and up to 3 channel doublings can be chosen. It is therefore encoded as a string of 6 values, with each value assigned to one of the possible combinatorial choices: nothing, downsampling, double channels or downsampling+double channels. If the sampled architecture is smaller than the maximum size 6, the genome is padded with 0s to reach 6 values.

B Search space encodings

In this section, we detail the encodings of the search spaces used for the experiments, largely based on the implementation from Krishnakumar et al. (2022).

B.1 NAS-Bench-201

The genome encoding of architectures from the NB201 space (Dong & Yang, 2020) is directly inferred from their graph-like cell structure. The cell is directly copied and stacked to form a complete network, therefore it is sufficient to encode a single cell. Each edge of the cell's graph is assigned to a genome position. The operation choices of the space (5 possible operations) are encoded into each position as a numerical value. With 6 edges in the graph, the resulting genome is a 6 values long string.

B.2 TransNAS-Bench-101 Micro

Due to being based directly on the same setting, the genome encoding of architectures for the TNB101-Micro space (Duan et al., 2021) is similar to the encoding for the NB201, with 6 edges and 4 operation choices.

B.3 TransNAS-Bench-101 Macro

The TNB101-Macro space (Duan et al., 2021) consists in a stack of 4 to 6 residual blocks where the locations of up to 4 downsamplings and up to 3 channel doublings can be chosen. It is therefore encoded as a string of 6 values, with each value assigned to one of the possible combinatorial choices: nothing, downsampling, double channels or downsampling+double channels. If the sampled architecture is smaller than the maximum size 6, the genome is padded with 0s to reach 6 values.

C ZCP details

In this section, we present the definitions of each benchmarked ZCP based on theoretical insights and their implementation.

C.1 `#params`

`#params` is defined simply as the number of trainable parameters in the network. We directly use the network implementation to count its parameters without need for additional data.

C.2 `jacov`

`jacov` is introduced in Mellor et al. (2021) as the standalone method NASWOT then later renamed as `jacov` in Abdelfattah et al. (2021). It is defined based on the covariance matrix of the Jacobian:

$$C = cov(\nabla_w(\mathcal{L}))$$

where $\mathcal{L}$ is the loss. The Jacobian is obtained by concatenation of the gradients of the model from a backward pass over a single batch of data. Then, `jacov` is constructed as a measure of the ability of the model to differentiate inputs in the batch:

$$\texttt{jacov} = -\sum_{i=1}^{n} \left(log(\lambda^i) + \frac{1}{\lambda^i} \right)$$

where $\lambda_1 \dots \lambda_n$ are the eigenvalues of C.

C.3 `grad_norm`

`grad_norm` is introduced by Abdelfattah et al. (2021) as a naive baseline for ZCPs. When the norm of the gradients of the model is higher, the model intuitively progresses faster in its parameter space, and is therefore easier to train.

$$\texttt{grad_norm} = \|\nabla_w(\mathcal{L}(m, w)\|_{\mathcal{F}}$$

C.4 `snip`, `grasp`, `synflow`

`snip` (Lee et al., 2018) `grasp` (Wang et al., 2020) and `synflow` (Tanaka et al., 2020) are saliency metrics introduced in the pruning literature to determine the importance of specific parameters. They are aggregated over the entire network to provide a measure of the overall saliency of parameters in the model.

$$\texttt{snip} = \sum_{i}^{N} |\frac{\partial \mathcal{L}}{\partial \theta_i} \odot \theta_i|$$

$$\texttt{grasp} = \sum_{i}^{N} - \left(H \frac{\partial \mathcal{L}}{\partial \theta_i} \right) \odot \theta_i$$

$$\texttt{synflow} = \sum_{i}^{N} |\frac{\partial \mathcal{L}}{\partial \theta_i} \odot \theta_i|$$

All 3 metrics are summed over the N layers of the network. θ_i are the trainable parameters at layer i, $\mathcal{L}$ is the loss and $\odot$ is the Hadamard product. In the case of `grasp`, the Hessian H is computed via the $\mathcal{R}\{\cdot\}$ trick (Pearlmutter, 1994). `synflow` is similar to `snip`, but is data-agnostic as it does not compute over a batch of data, instead using a full matrix of 1 as input to estimate the synaptic flow.

C.5 `fisher`

`fisher` is introduced as a ZCP by Abdelfattah et al. (2021). It is based on the aggregated sum of layerwise Fisher information (Theis et al., 2018; Turner et al., 2019) over the N layers of the model:

$$\texttt{fisher} = \sum_{i}^{N} \left(\frac{\partial \mathcal{L}}{\partial z_i} z_i \right)^2$$

where z_i is the aggregation of post-activations at layer i and $\mathcal{L}$ is the loss.

C.6 `ntk`

`ntk` is introduced as a ZCP in the TE-NAS method (Chen et al., 2021) where it is paired with the number of linear regions. It is defined as the condition number of the Neural Tangent Kernel (NTK) (Jacot et al., 2018; Lee et al., 2019; Xiao et al., 2019). Networks with lower condition number have better convergence property and trainability. We estimate the Neural Tangent Kernel following the TE-NAS estimation scheme:

$$\Theta = \nabla_w (\mathcal{L}^T \nabla_w (\mathcal{L})$$

where $\mathcal{L}$ is the loss and w are the trainable parameters of the model. Considering the eigenvalues $\lambda_0 \dots \lambda_n$ of Θ, we have:

$$\texttt{ntk} = \lambda_n / \lambda_0$$

C.7 `zen`

`zen` is introduced in Lin et al. (2021) as a ZCP based on the noise error of the model and the statistics of the batch normalization layers (BN). Therefore, it is specific to search spaces that contain batch normalizations.

Let x, ϵ the inputs of `zen` sampled from a normal distribution $\mathcal{N}(0, 1)$. For model f and hyperparameter α, we have:

$$\Delta = \mathbb{E}_{x,\epsilon} \| f(x) - f(x + \alpha\epsilon) \|_F$$

And for all BN layer in the model, we define:

$$\bar{\sigma} = \textstyle\sum_i log \left(\sqrt{\sum_j^m \sigma_{i,j}^2 / m} \right)$$

which considers the standard deviation $\sigma_{i,j}$ of BN layer i at the jth out of m channels.

Both parts of the metric are then summed:

$$\texttt{zen} = log(\Delta) + \bar{\sigma}$$

C.8 `zico`

`zico` (Li et al., 2023) is introduced to evaluate the generalization capacity of architectures (Keskar et al., 2016; Liang et al., 2017) based on the statistics of the gradients:

$$\texttt{zico} = \sum_{i=1}^{N} log \left(\sum_{\omega \in \theta_i} \frac{\mathbb{E} \| \nabla_\omega \mathcal{L} \|}{\sqrt{Var(\nabla_\omega \mathcal{L})}} \right)$$

where θ_i are the trainable parameters at layer i out of N layers, and $\mathcal{L}$ is the loss.

C.9 `meco`

`meco` (Jiang et al., 2023) is introduced based on similar insights as `zico` by noticing that the Pearson correlation matrix is closely related to the Gram matrix. Let the Pearson correlation matrix over the outputs of mode f with input X:

$$[\mathcal{P}(X)]_{i,j} = \frac{\mathbb{E}\|(f(X_i) - \mu_{f(X_i)})(f(X_j) - \mu_{f(X_j)})\|}{\sigma_{f(X_i)}\sigma_{f(X_j)}}$$

where μ_x and σ_x are the mean and standard deviation of x, respectively. `meco` is then computed based on the minimum eigenvalue λ_{min} of the Pearson correlation matrix:

$$\texttt{meco} = \sum_{i=1}^{N} \lambda_{min}(\mathcal{P}(X))$$

C.10 `swap`

`swap` (Peng et al., 2024) is a reframing of the concept of linear regions partition in the latent space of the network to measure the expressivity of the network (Raghu et al., 2017; Xiong et al., 2020; Chen et al., 2021).

The set of sample-wise activation patterns of the network is gathered as:

$$\hat{\mathbb{A}}_\theta = \left\{ \mathbb{1}(p_s^{(v)})_{s=1}^{S}, v \in \{1, \dots, V\} \right\}$$

where $p_s^{(v)}$ is the post-activation of sample s at value v. $\hat{\mathbb{A}}_\theta$ contains the one-hot encodings of all activation patterns obtained in the network for the samples in batch S.

`swap` is then equal to the number of activation patterns, which is the cardinal of the set:

$$\texttt{swap} = |\hat{\mathbb{A}}_\theta|$$

D More details about the Spearman correlation benchmarks

In the main paper, confidence intervals for the Spearman correlation benchmarks are omitted to keep the table readable. The displayed value is averaged over 3 seeds. In the following tables we present the Spearman correlation for both the full space setting and the top 1% with 1σ error.

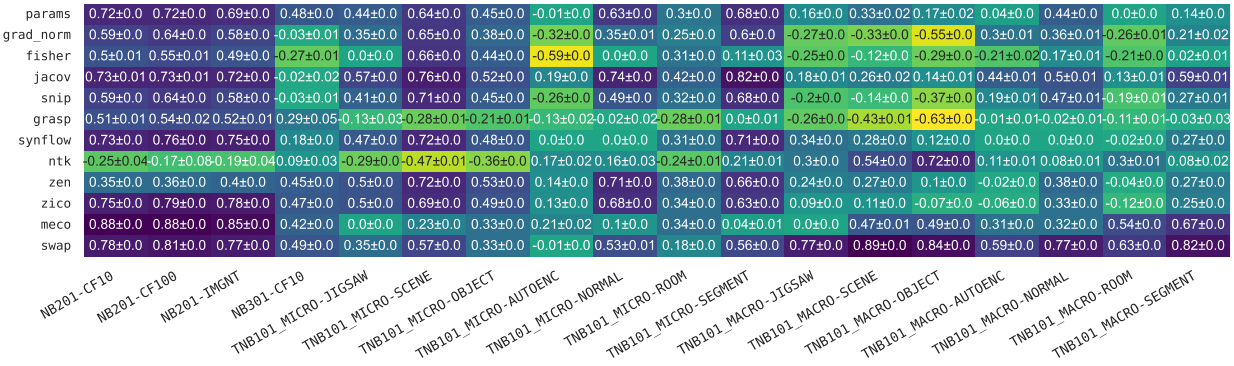


Figure 7: Spearman rank correlations of various ZCPs on the NAS-Bench-201, NAS-Bench-301 and TransNAS-Bench-101 benchmarks. Spearman was computed over 3 seeds against the entire search space.

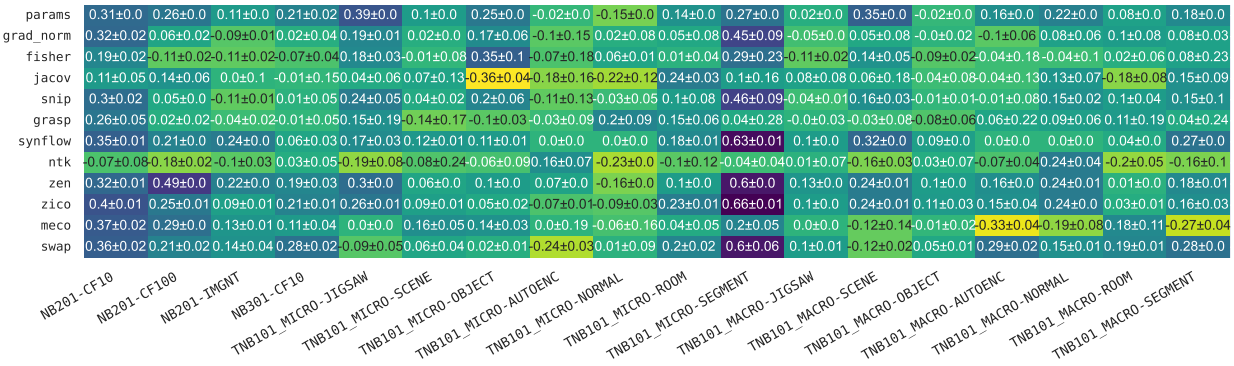


Figure 8: Spearman rank correlations of various ZCPs on the NB201, NB301 and TNB101 benchmarks. Spearman was computed on 3 seeds against the top 1% architectures in the space.

E Full ParaDis algorithm

Algorithm 1 **Para**meter **Dis**tribution Shift NAS (**ParaDis**)

Require: : population size P, sample size $p < P/2$, number of generations N, evolution steps per generation S, zero-cost metric $\mathcal{M}$, temperature schedule $[\tau_0, \tau_1, \ldots, \tau_N]$

population $\leftarrow \emptyset$
while *population.size* $< P$ **do**
    Randomly sample *arch* from the space
    $arch.score \leftarrow \mathcal{M}(arch)$
    $arch.params \leftarrow CountParams(arch)$
    Add *arch* to *population*
end while
μ, $\sigma \leftarrow Mean(population.params), STD(population.params)$
for *arch* in *population* **do**
    $arch.\Phi \leftarrow e^{-(\frac{arch.params-(\mu+\tau_0\sigma)}{\sigma})^2}$
end for
for $n = 1, 2, \ldots, N$ **do**
    $\tau \leftarrow \tau_n$
    for $s = 1, 2, \ldots S$ **do**
        $candidates_A$, $candidates_B \leftarrow p$, p random *archs* of *population*
        $candidates_C \leftarrow$ all remaining *archs* in *population*
        for $candidates_A$, $candidates_B$ **do**
            discarded $\leftarrow$ worst *arch* by Φ
            parent $\leftarrow$ best *arch* by *arch.score* except *discarded*
            child $\leftarrow Mutate(parent)$
            child.score $\leftarrow \mathcal{M}(child)$
            child.params $\leftarrow CountParams(child)$
            Replace *discarded* with *child* in *population*
        end for
        $discarded_C \leftarrow$ worst *arch* by Φ in $candidates_C$
        $child_C \leftarrow Crossover(parent_A, parent_B)$
        $child_C.score \leftarrow \mathcal{M}(child_C)$
        $child_C.params \leftarrow CountParams(child_C)$
        Replace $discarded_C$ with $child_C$ in *population*
    end for
    μ, $\sigma \leftarrow Mean(population.params), STD(population.params)$
    for *arch* in *population* **do**
        $arch.\Phi \leftarrow e^{-(\frac{arch.params-(\mu+\tau\sigma)}{\sigma})^2}$
    end for
end for